Loop

++x	Variable x wird um 1 erhöht
−−x	Wenn x = 0 ist, geschieht nichts. Ansonsten wird x um 1 erniedrigt.
stop	Stoppt das Programm
loop x ⟨Programm⟩ *end*	Führt wiederholt ⟨Programm⟩ aus, und zwar genau so oft, wie der Wert in x am Anfang war.

ersetze *loop* durch *while*

While

++x	Variable x wird um 1 erhöht
−−x	Wenn x = 0 ist, geschieht nichts. Ansonsten wird x um 1 erniedrigt.
stop	Stoppt das Programm
while x ⟨Programm⟩ *end*	Führt wiederholt ⟨Programm⟩ aus, solange x ≠ 0

▸ **loop-Schleifen in extended While**

loop x
⟨P⟩
end

```
h ← x
while h
   ⟨P⟩
   −−h
end
```

⇦ *h komme in ⟨P⟩ nicht vor*

▸ **Beobachtung 1:** Jedes Loop-Programm kann in ein While-Programm überführt werden.

▸ **Beobachtung 2:** … aber nicht umgekehrt!

▸ **Beispiel**

```
while x
   ++x
end
```

While-Programme können sich aufhängen, Loop-Programme nicht.

▶ **Partielle Funktionen**

```
while x
   ++x
end
return x
```

⇦ nur für x = 0 definiert

Loop

Loop-Programme…
…akzeptieren Sprachen ⊆ Σ*
…sind selbst Text ∈ Σ*

Seien p und s ∈ Σ*.
Man kann sicher und in endlicher Zeit entscheiden:

1. ist p ein Loop-Programm?
2. akzeptiert das Loop-Programm p die Eingabe s?

▶ **Loop-Programme, die sich nicht selbst akzeptieren**

L = {p ∈ Σ* | p ist ein Loop-Programm und
p akzeptiert *nicht* die Eingabe s=p}

»die Menge aller Loop-Programme,
die sich selbst nicht akzeptieren«

Frage: Gibt es ein Loop-Programm, das L akzeptiert?
Antwort: Nein!

Frage: Gibt es ein Python- oder Java-Programm,
das L akzeptiert?
Antwort: Ja!

Eingabe p:

1. prüfe, ob p ein Loop-Programm ist.
2. Wenn ja:
   - *simuliere* das Loop-Programm p
     mit Eingabe p.
   - p akzeptiert p ⇒ lehne p ab
   - p lehnt p ab ⇒ akzeptiere p

▶ **Dasselbe für While?**

L = {p ∈ Σ* | p ist ein While-Programm und
p akzeptiert *nicht* die Eingabe s=p}

»die Menge aller While-Programme,
die sich selbst nicht akzeptieren«

⇒ L ist aber gar nicht berechenbar!

▸ neuer Loop-Befehl

hangup Das Loop-Programm hängt sich auf und endet nie.

▸ Beispiel

```
while x
   ++x
end
return x
```

```
if x
   hangup
end
return x
```

⇦ auch nur für x = 0 definiert

▸ Ackermann-Funktion

$A_k(n)$
- n: Eingabegröße
- k: Stufe

Für $k \in \mathbb{N}$ und $n \in \mathbb{N}$ mit $n \geq 2$ gelte:

$A_0(1) = 2$
$A_k(1) = 2$
$A_0(n) = n+2$
$A_k(n) = A_{k-1}(A_k(n-1))$

$$
\begin{aligned}
A_k(n) &= A_{k-1}(A_k(n-1)) \\
&= A_{k-1}(A_{k-1}(A_k(n-2))) \\
&= A_{k-1}(A_{k-1}(A_{k-1}(A_k(n-3)))) \\
&= A_{k-1}(A_{k-1}(A_{k-1}(\dots A_{k-1}(\underbrace{A_k(1)}_{2})))))
\end{aligned}
$$

$$A_k(n) = \underbrace{A_{k-1}(A_{k-1}(A_{k-1}(\dots A_{k-1}}_{n-1 \text{ mal}}(2))))$$

$$A_0(n) = \begin{cases} 2, & \text{wenn } n = 1 \\ n+2, & \text{wenn } n \geq 2 \end{cases}$$

$A_1(n) = 2n$

$A_2(n) = 2^n$

$A_3(n) = \underbrace{2^{2^{\cdots^2}}}_{n}$

$A_3(1) = 2$
$A_3(2) = 2^2 = 4$
$A_3(3) = 2^{2^2} = 2^4 = 16$
$A_3(4) = 2^{2^{2^2}} = 2^{16} = 65536$
$A_3(5) = 2^{2^{2^{2^2}}} = 2^{65536} > 2 \cdot 10^{19729}$ 😨

$\underbrace{2003529930406846464979072351560255750447\ldots56736}_{19729 \text{ Dezimalstellen}}$

$A_k(n)$

```
a ← 2
loop n−1
   a ← A_{k−1}(a)
end
```

k ineinander geschachtelte *loop*-Schleifen

Beobachtung: Zur Berechnung von $A_k(n)$ braucht man
k geschachtelte *loop*-Schleifen.

Jedes Loop-Programm hat eine maximale Schachtelungstiefe von *loop*-Schleifen.

⇒ es gibt kein Loop-Programm, das $A_k(n)$ für beliebige k und n ausrechnen kann.

Aber: Es gibt ein While-Programm, das $A_k(n)$ für beliebige k und n ausrechnen kann.

Fazit:

While kann mehr als Loop